

Flowchart Logic and Backend Functions Breakdown

1) Flowchart Logic نصي

School Timetable AI Engine Flow

اكتب منطق المحرك بالتسلسل التالي:

Start

1. Initialize Generation Request

- استقبال طلب توليد الجدول
- حدّد:
 - school_id
 - academic_year_id
 - term_id
 - generation_mode
 - initiated_by

2. Load Active Timetable Context

- اطلب إعدادات الجدول الخاصة بالمدرسة
- اطلب الأيام الدراسية المفعلة
- اطلب الفترات الزمنية
- اطلب الفترات التعليمية فقط
- اطلب الفصول النشطة
- اطلب الصفوف
- اطلب المواد المطلوبة لكل صف
- اطلب المعلمين وربطهم بالمواد
- اطلب قيود توفر المعلمين
- اطلب قيود توفر الفصول
- اطلب قواعد النصاب
- اطلب تفضيلات التوليد
- اطلب قيود الغرف إذا كانت مفعلة

Decision:

Fail: Missing Context إذا فشل تحميل أي جزء أساسي → انتقل إلى

3. Run Readiness Validation

تحقق من:

- هل يوجد عام دراسي نشط؟
- هل يوجد فصل دراسي نشط؟
- هل توجد أيام دراسية مفعلة؟
- هل توجد Teaching Slots؟
- هل توجد فصول؟
- هل توجد مواد مرتبطة بالصفوف؟
- هل يوجد عدد حصص أسبوعية لكل مادة؟
- هل يوجد معلم صالح لكل مادة مطلوبة؟
- هل قواعد النصاب معروفة؟
- هل الإعدادات الزمنية منطقية؟

Decision:

- Return Readiness Errors → إذا كانت البيانات غير مكتملة
- إذا كانت صالحة → انتقل للخطوة التالية

4. Create Generation Run Record

- أنشئ سجل تشغيل جديد في:
 - timetable_generation_runs
- الحالة الأولية:
 - status = running

5. Create Draft Timetable Version

- أنشئ نسخة جديدة في:
 - timetable_versions
- الحالة:
 - status = draft

6. Build Scheduling Context Model

- كَوّن نموذجًا داخليًا موحدًا يحتوي على:
 - active weekdays
 - teaching slots
 - class map
 - grade-subject map
 - teacher-subject map
 - workload rules
 - availability map
 - constraints map
 - preferences map

7. Build Demand Units

- لكل فصل:
 - اجلب المواد الخاصة بصفه
 - لكل مادة:
 - اجلب عدد الحصص الأسبوعية المطلوبة
 - Scheduling Units أنشئ عددًا مساويًا من

Output:

قائمة كاملة من وحدات الجدولة

8. Prioritize Demand Units

- رتب الوحدات من الأصعب إلى الأسهل بناءً على:
 - عدد المعلمين المتاحين
 - عدد الفترات المحتملة
 - أولوية المادة
 - كثافة الفصل
 - الحاجة لغرفة خاصة
 - قيود التوقيت
 - قرب المعلم من سقف النصاب

9. Initialize Occupancy State

- أنشئ حالة إشغال داخلية:
 - إشغال المعلمين
 - إشغال الفصول
 - إشغال الغرف
 - عدادات المادة اليومية
 - عدادات النصاب اليومي
 - عدادات النصاب الأسبوعي

Main Assignment Loop

10. Pick Next Highest-Priority Unit

- اختر أعلى وحدة أولوية لم يتم إسنادها بعد

11. Build Candidate Slots

- أنشئ قائمة الفترات المحتملة لهذه الوحدة

محتمل Slot لكل

- تحقق من:
 - teacher availability
 - class availability
 - teaching slot validity
 - workload rules

- subject daily max
- room requirement
- forbidden timing constraints

Decision:

Fallback Handling صالح → انتقل إلى Slot إذا لا يوجد أي

12. Score Candidate Slots

- بناءً على Slot لكل Score احسب:
 - ملائمة الوقت
 - تفضيلات المادة
 - تفضيلات المعلم
 - تقليل الفجوات
 - توازن الحمل
 - تقليل التكرار
 - جودة تموضع المادة
- من الأعلى إلى الأقل Slots رتب الـ

13. Try Assign Best Slot

- خذ أعلى Slot
- Hard Constraints تحقق مرة أخيرة من
- إذا كان صالحًا:
 - timetable_entry أنشئ
 - Occupancy State حدّث
 - assigned علّم الوحدة كـ

Decision:

- إذا نجح → ارجع إلى الخطوة 10
- التالي Slot إذا فشل → جرّب الـ

Fallback Handling

14. Try Alternative Teacher

- إذا كانت الوحدة لها أكثر من معلم مرشح
 - باستخدام معلم آخر Candidate Slots أعد بناء

Decision:

Assign → إذا نجح
إذا فشل → تابع

15. Try Swap Strategy

- ابحث عن حصة أقل أولوية يمكن تحريكها
- Hard Constraints آمن لا يكسر swap اختبر

Decision:

إذا نجح → Assign

إذا فشل → تابع

16. Try Limited Backtracking

- فك آخر مجموعة صغيرة من الإسنادات المرتبطة بالتعارض
- أعد محاولة الإسناد

Decision:

إذا نجح → Assign

إذا فشل → تابع

17. Relax Soft Constraints

- فقط Soft Constraints اسمح بتقليل درجة الالتزام بالـ
- أعد بناء Candidate Slots

Decision:

إذا نجح → Assign

إذا فشل → تابع

18. Log Unresolved Unit

- سجّل مشكلة في:
 - timetable_generation_issues
- النوع:
 - unassigned_unit
- انتقل للوحدة التالية

End of Main Loop

19. Check If All Units Processed

Decision:

- إذا لا → عُد إلى الخطوة 10
- إذا نعم → أكمل

20. Run Optimization Pass

- حلّ الجدول الناتج
- ابحث عن:
 - فجوات المعلمين
 - فجوات الفصول
 - تكدس المواد
 - اختلال الأحمال
 - ضعف التوزيع اليومي
 - تموضع غير جيد للمواد الأساسية

- نَفِّذ تحسينات محلية:
 - move
 - swap
 - rebalance

21. Evaluate Final Quality

- احسب:
 - completion_score
 - hard_conflict_score
 - teacher_balance_score
 - class_balance_score
 - subject_distribution_score
 - soft_constraint_score
 - quality_score

22. Update Draft Version

- حدِّث:
 - quality_score
 - warnings_count
 - conflicts_count
 - notes

23. Update Generation Run

- حدِّث:
 - assigned_units
 - unassigned_units
 - hard_conflicts
 - soft_conflicts
 - status = completed
 - finished_at

24. Return Final Result

أعد للمستخدم:

- version_id
- generation_status
- quality_score
- assigned_units
- unassigned_units
- warnings
- issues_summary

Failure Paths

Fail: Missing Context

- لا تنشئ جدول
- أعد رسالة خطأ واضحة
- سجّل المشكلة إن لزم

Fail: Readiness Validation Failed

- أعد قائمة تفصيلية بالنواقص
- لا تبدأ التوليد

Fail: Runtime Generation Error

- حدّث generation_run.status = failed
- خزّن error details
- لا تنشر أي نسخة

2) Backend Functions Breakdown

Timetable Engine Backend Modules

سأقسمها لموديولات ووظائف واضحة.

Module A: Context Loader

1. load_timetable_context(school_id, academic_year_id, term_id)

المسؤولية:

تحميل كل البيانات اللازمة للتوليد من قاعدة البيانات.

يجب أن يجلب:

- school settings
- weekdays
- time blocks
- teaching slots
- grades
- classes
- subjects
- grade_subjects
- teachers
- teacher_subjects
- teacher_workload_rules
- teacher_unavailability
- class_unavailability

- subject_time_preferences
- room requirements
- generation preferences

output:

```
{
  "school": {},
  "settings": {},
  "weekdays": [],
  "time_blocks": [],
  "teaching_slots": [],
  "classes": [],
  "subjects": [],
  "grade_subjects": [],
  "teachers": [],
  "teacher_subjects": [],
  "constraints": {},
  "preferences": {}
}
```

Module B: Readiness Validator

2. validate_timetable_readiness(context)

المسؤولية:

فحص جاهزية التوليد قبل البدء.

checks:

- active year exists
- active term exists
- teaching days exist
- teaching slots exist
- classes exist
- grade-subject mappings exist

- weekly requirements exist
- each required subject has candidate teachers
- workload rules valid
- timetable settings valid

output:

```
{
  "is_valid": True,
  "issues": []
}
```

Module C: Version & Run Manager

3. `create_generation_run(context, initiated_by, generation_mode)`

.ينشئ سجل تشغيل جديد.

4. `create_draft_timetable_version(context, initiated_by, generation_mode)`

.جديدة Draft ينشئ نسخة

5. `mark_generation_run_failed(run_id, error_payload)`

failed. يحدث حالة التشغيل إلى

6. `mark_generation_run_completed(run_id, stats)`

completed. يحدث حالة التشغيل إلى

7. `update_timetable_version_metrics(version_id, metrics)`

:يحدث

- quality_score
- warnings_count
- conflicts_count

Module D: Scheduling Context Builder

8. `build_scheduling_model(context)`

المسؤولية:

سريعة وملائمة للخوارزمية Structures تحويل البيانات الخام من قاعدة البيانات إلى

مثل:

- teacher_availability_map
- class_availability_map
- subject_teacher_map
- grade_subject_requirements_map
- daily_slot_map
- teacher_workload_map
- room_availability_map

Module E: Demand Unit Builder

9. build_demand_units(context, scheduling_model)

المسؤولية:

إنشاء وحدات الجدولة الفعلية من الاحتياج الأسبوعي.

output:

مثل Units قائمة

```
[  
  {  
    "unit_id": "...",  
    "class_id": 12,  
    "grade_id": 3,  
    "subject_id": 8,  
    "candidate_teacher_ids": [15, 17],  
    "weekly_index": 1,  
    "priority_weight": 92  
  }  
]
```

10. prioritize_demand_units(units, scheduling_model)

المسؤولية:

ترتيب الوحدات حسب الصعوبة

sorting factors:

- fewest candidate teachers
- fewest candidate slots
- core subject priority
- weekly frequency
- special room requirement
- teacher load tightness

Module F: Occupancy State Manager

11. initialize_occupancy_state(context, version_id)

المسؤولية:

بناء الحالة الداخلية للإشغال

tracks:

- teacher busy slots
- class busy slots
- room busy slots
- teacher daily load
- teacher weekly load
- class daily subject counts

12. mark_slot_assigned(occupancy_state, unit, slot)

المسؤولية:

تحديث حالة الإشغال بعد الإسناد

13. mark_slot_unassigned(occupancy_state, unit, slot)

المسؤولية:

backtracking أو swap فك الإسناد أثناء

Module G: Candidate Slot Builder

14. build_candidate_slots(unit, scheduling_model, occupancy_state)

المسؤولية:

إنتاج كل الفترات المحتملة لوحدة واحدة

loop dimensions:

- weekday
- time_block
- teacher_id

filter by hard constraints

output:

```
[  
  {  
    "weekday_number": 1,  
    "time_block_id": 5,  
    "teacher_id": 15  
  }  
]
```

15. passes_hard_constraints(unit, slot, scheduling_model, occupancy_state)

يتحقق من:

- teacher not busy
- class not busy
- room valid
- time allowed
- workload not exceeded
- subject daily max not exceeded
- slot is teaching slot

Module H: Slot Scoring Engine

16. score_candidate_slots(unit, candidate_slots, scheduling_model, occupancy_state)

المسؤولية:

وترتيب النتائج Slot لكل Score حساب

17. score_single_slot(unit, slot, scheduling_model, occupancy_state)

score components:

- teacher availability
- class continuity
- reduce gaps
- preferred subject time
- preferred teacher time
- daily load balance
- avoid consecutive repetition
- avoid late heavy subjects

output:

```
{  
  
  "slot": {...},  
  
  "score": 84  
}
```

Module I: Assignment Engine

18. assign_unit_to_best_slot(unit, scored_slots, version_id, occupancy_state, scheduling_model)

المسؤولية:

إلى الأقل Score تجربة الإسناد من الأعلى

steps:

- recheck hard constraints
- insert timetable entry
- update occupancy state
- return success/failure

19. create_timetable_entry(version_id, unit, slot)

المسؤولية:

timetable_entries حفظ الحصة الفعلية داخل

Module J: Fallback Engine

20. `try_alternative_teachers(unit, scheduling_model, occupancy_state)`

.يجرب معلمين آخرين.

21. `try_swap_strategy(unit, version_id, scheduling_model, occupancy_state)`

.يجرب تبديلاً آمناً مع حصة أقل أولوية.

22. `try_limited_backtracking(unit, version_id, scheduling_model, occupancy_state)`

.يفك عددًا محدودًا من الإسنادات ويعيد المحاولة.

23. `try_relaxed_soft_constraints(unit, scheduling_model, occupancy_state)`

Soft Constraints. يعيد التقييم مع مرونة أكبر في

24. `resolve_unassigned_unit(unit, version_id, scheduling_model, occupancy_state)`

.يشغل كل الاستراتيجيات السابقة بالتسلسل.

Module K: Optimization Engine

25. `run_post_generation_optimization(version_id, scheduling_model, occupancy_state)`

المسؤولية:

.تحسين الجدول بعد الإسناد الأولي.

sub-functions:

- تقليل فجوات المعلمين
- تقليل فجوات الفصول
- إعادة توزيع المواد
- موازنة الأحمال
- تحسين تموضع المواد الأساسية

26. optimize_teacher_gaps(version_id, scheduling_model, occupancy_state)

27. optimize_class_gaps(version_id, scheduling_model, occupancy_state)

28. rebalance_subject_distribution(version_id, scheduling_model, occupancy_state)

29. improve_core_subject_placement(version_id, scheduling_model, occupancy_state)

Module L: Quality Evaluation

30. evaluate_timetable_quality(version_id, scheduling_model, occupancy_state)

يحسب:

- completion_score
- hard_conflict_score
- teacher_balance_score
- class_balance_score
- subject_distribution_score
- soft_constraint_score
- total_quality_score

31. count_unassigned_units(units, assigned_units)

32. detect_remaining_conflicts(version_id)

33. build_quality_summary(metrics)

Module M: Diagnostics & Logging

34. log_generation_issue(run_id, issue_type, severity, entity_type, entity_id, message, details)

يسجل أي مشكلة.

35. log_timetable_audit(version_id, action_type, action_by, target_type, target_id, old_value, new_value)

يسجل أي تعديل أو قرار مهم.

36. build_generation_diagnostics(run_id, version_id)

تشخيصية نهائية Summary يرجع

Module N: Main Orchestrator

37. generate_timetable(school_id, academic_year_id, term_id, generation_mode, initiated_by)

هذه هي الوظيفة الأساسية التي تربط كل شيء

التسلسل:

1. load context
2. validate readiness
3. create run
4. create draft version
5. build scheduling model
6. build demand units
7. prioritize units
8. initialize occupancy
9. assign units iteratively
10. resolve failed assignments
11. optimize final timetable
12. evaluate quality
13. update version metrics
14. complete run
15. return result

3) Orchestrator Pseudocode

```
def generate_timetable(school_id, academic_year_id, term_id, generation_mode, initiated_by):
```

```
    context = load_timetable_context(school_id, academic_year_id, term_id)
```

```
    readiness = validate_timetable_readiness(context)
```

```
    if not readiness["is_valid"]:
```

```
return {  
    "status": "failed",  
    "reason": "readiness_validation_failed",  
    "issues": readiness["issues"]  
}
```

```
run = create_generation_run(context, initiated_by, generation_mode)
```

```
version = create_draft_timetable_version(context, initiated_by, generation_mode)
```

```
try:
```

```
    scheduling_model = build_scheduling_model(context)
```

```
    units = build_demand_units(context, scheduling_model)
```

```
    units = prioritize_demand_units(units, scheduling_model)
```

```
    occupancy_state = initialize_occupancy_state(context, version["id"])
```

```
    for unit in units:
```

```
        candidate_slots = build_candidate_slots(unit, scheduling_model, occupancy_state)
```

```
        scored_slots = score_candidate_slots(unit, candidate_slots, scheduling_model,  
occupancy_state)
```

```
        assigned = assign_unit_to_best_slot(  
            unit, scored_slots, version["id"], occupancy_state, scheduling_model
```

```
        )
```

```
    if not assigned:
```

```
        assigned = resolve_unassigned_unit(  
            unit, version["id"], scheduling_model, occupancy_state
```

```
            unit, version["id"], scheduling_model, occupancy_state
```

)

if not assigned:

```
log_generation_issue(  
    run["id"],  
    "unassigned_unit",  
    "high",  
    "scheduling_unit",  
    None,  
    "Could not assign this unit to any valid slot",  
    unit  
)
```

run_post_generation_optimization(version["id"], scheduling_model, occupancy_state)

quality = evaluate_timetable_quality(version["id"], scheduling_model, occupancy_state)

update_timetable_version_metrics(version["id"], quality)

mark_generation_run_completed(run["id"], quality)

return {

"status": "success",

"version_id": version["id"],

"quality_score": quality["total_quality_score"],

"summary": quality

}

except Exception as e:

```
mark_generation_run_failed(run["id"], {"error": str(e)})
```

```
raise
```

4) Recommended Service Structure

اجعله هكذا services، لو المشروع منظم على أسلوب

/services/timetable/

context_loader.py

readiness_validator.py

scheduling_model_builder.py

demand_unit_builder.py

prioritization_service.py

occupancy_state.py

candidate_slot_service.py

hard_constraints.py

scoring_engine.py

assignment_engine.py

fallback_engine.py

optimization_engine.py

quality_evaluator.py

diagnostics_service.py

generation_orchestrator.py

5) الصياغة المختصرة

Build the timetable backend as modular services.

The engine should include:

- context loading
- readiness validation
- draft version creation
- generation run tracking
- scheduling model construction
- demand unit generation

- difficulty-based prioritization
- candidate slot generation
- hard-constraint validation
- slot scoring
- iterative assignment
- fallback resolution
- post-generation optimization
- quality evaluation
- diagnostics and audit logging
- orchestration through a main generation service

Each responsibility must be isolated in a dedicated backend function or service for maintainability, debuggability, and future scaling.